



KLE Technological University

Creating Value,
Leveraging Knowledge

A Mini Project report on

Knowledge Based Enhanced Retrieval Augmented Generation

Submitted

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

IN

Department of COMPUTER SCIENCE AND ENGINEERING

Submitted By

Sneha Chinchakandi	01FE23BCS377
Yashaswini S M	01FE23BCS288
Mahesh Khatwate	01FE23BCS291
Amogh Annigeri	01FE23BCS210

Under the guidance of

Dr.Meena S M

School of Computer Science and Engineering

KLE Technological University, Hubballi

2025-2026



KLE Technological University

Creating Value,
Leveraging Knowledge

Department of Computer Science and Engineering

2025-2026

CERTIFICATE

This is to certify that project entitled KNOWLEDGE GRAPH ENHANCED RETRIEVAL-AUGMENTED GENERATION is a bonafied work carried out by the student team Sneha Chinchakandi - 01FE23BCS377, Yashaswini S M - 01FE23BCS288, Mahesh Khatawate - 01FE23BCS291, Amogh Annigeri - 01FE23BCS210, in partial fulfillment of the completion of 5th semester Mini Project during the year 2025 – 2026. The project report has been approved as it satisfies the academic requirement with respect to the project work prescribed for the above said course.

Guide Name
Dr.Meena S M

Head, CSE
Dr. Vijaylakshmi M.

External Viva-Voce

Name of the examiners

Signature with date

1 _____

2 _____

ABSTRACT

Retrieval-Augmented Generation (RAG) improves the factual accuracy of Large Language Models by retrieving documents before generating answers. However, traditional RAG systems rely mainly on unstructured text and often fail on complex multi-hop questions where information is spread across multiple sources. They also retrieve excessive and sometimes irrelevant content, which reduces precision and may lead to weakly supported or hallucinated answers.

This project presents a Knowledge Graph Enhanced RAG (KG-RAG) framework that integrates structured knowledge graphs with neural retrieval and generation. The system follows a multi-stage, precision-focused pipeline: broad initial retrieval to maximize recall, knowledge-graph-based expansion to capture related entities, and neural reranking to remove noisy information. A deduplication mechanism limits supporting facts to the most relevant ones, and the language model is constrained to generate answers using only the retrieved context.

The proposed approach is evaluated on the HotpotQA dataset using metrics such as Exact Match (EM), F1 score, Supporting Fact Precision, and Joint Precision. Results show significant improvements over the baseline, with Supporting Fact Precision increasing by 86.48% and Joint Precision improving by 96.18%. These findings demonstrate that incorporating knowledge graphs into RAG systems improves multi-hop reasoning, reduces noise, and produces more reliable, well-supported answers, making the framework suitable for high-trust applications such as education.

Keywords : *KRetrieval-Augmented Generation, Knowledge Graph, Multi-hop Question Answering, Neural Reranking, Information Filtering.*

ACKNOWLEDGEMENT

We would like to thank our faculty and management for their professional guidance towards the completion of the mini project work. We take this opportunity to thank Dr. Ashok Shettar, Pro-Chancellor, Dr. P.G Tewari, Vice-Chancellor and Dr. B.S.Anami, Registrar for their vision and support.

We also take this opportunity to thank Dr. Meena S.M, Professor and Dean of Faculty, SoCSE, Dr. Vijayalakshmi M, Professor, Head of CSE for having provided us direction and facilitated for enhancement of skills and academic growth.

We thank Dr.Meena S M and SoCSE for the constant guidance during interaction and reviews.

We extend our acknowledgment to the reviewers for critical suggestions and inputs. We also thank project coordinator Dr. Uday Kulkarni, Mr. Sunil V.Gurlahosur and reviewers for their suggestions during the course of completion. We express gratitude to our beloved parents for constant encouragement and support.

Sneha Chinchakandi - 01FE23BCS377

Yashaswini S M - 01FE23BCS288

Mahesh Khatawate - 01FE23BCS291

Amogh Annigeri - 01FE23BCS210

CONTENTS

ABSTRACT	i
CONTENTS	iii
LIST OF TABLES	iv
LIST OF FIGURES	v
1 INTRODUCTION	1
1.1 Preamble	1
1.2 Motivation	1
1.3 Literature Study	2
1.4 Problem Definition	3
1.5 Objectives of the project	3
2 SOFTWARE REQUIREMENT SPECIFICATION	4
2.1 Overview of SRS	4
2.2 Requirement Specifications	4
2.2.1 Functional Requirements	5
2.2.2 Use case diagram	5
2.2.3 Use Case descriptions using scenarios, strictly as per Pressman Template	7
2.2.4 Nonfunctional Requirements	12
2.2.5 Software and Hardware Requirement Specifications	12
3 PROPOSED SYSTEM	14
3.1 Description of Proposed System.	14
3.1.1 Description of Target Users	14
3.1.2 Advantages / Applications of Proposed System	15
3.1.3 Scope (Boundary of Proposed System)	16
4 SYSTEM DESIGN	17
4.1 Architecture of the System	17
4.2 Class Diagram	18
4.3 Sequence diagram	20
4.4 ER diagram and schema	21
4.5 State transition diagram	23

4.6	Data Set Description(if applicable)	24
5	IMPLEMENTATION	25
5.1	Proposed Methodology	25
5.2	Description of Modules	26
6	TESTING	29
6.1	Test Plan and Test Cases	29
6.1.1	Types of Testing Performed :	29
6.1.2	Sample Test Cases	30
7	RESULTS DISCUSSIONS	33
7.1	Evaluation Overview	33
7.2	Answer Quality Analysis	33
7.3	Supporting Fact and Explainability Analysis	34
7.4	Joint Metrics (Multi-hop Reasoning Performance)	34
8	CONCLUSIONS AND FUTURE SCOPE	35
8.1	Conclusion	35
8.2	Future Scope	35
	REFERENCES	38

LIST OF TABLES

2.1	Use Case Description for UC-1: Submit Query and Generate Answer	7
2.2	Use Case Description for UC-2: Knowledge Graph Expansion for Multi-hop Reasoning	8
2.3	Use Case Description for UC-3: Neural Reranking and Filtering	9
2.4	Use Case Description for UC-4: Answer Generation	10
2.5	Use Case Description for UC-5: Supporting Fact Extraction and Presentation .	11
7.1	Answer Quality Comparison	33
7.2	Supporting Fact Metrics	34
7.3	Joint Metrics	34

LIST OF FIGURES

2.1	Use Case Diagram of the Proposed KG-RAG System	6
4.1	Layered Architecture of the Proposed KG-RAG System	17
4.2	UML Class Diagram of the Proposed KG-RAG System	19
4.3	Sequence Diagram of the Proposed KG-RAG System	21
4.4	ER Diagram of the Proposed KG-RAG System	22
4.5	State Transition Diagram of the Proposed KG-RAG System	23
4.6	Dataset Description: Document Corpus and Knowledge Graph	24
5.1	Workflow of the Proposed KG-RAG Methodology	25

Chapter

1

INTRODUCTION

Sophisticated question-answering systems in recent years have been a significant component of educational, research, and decision-support applications. Such systems are supposed to deliver factual, dependable, and clarifiable responses to queries by users [1]. Due to the rapid evolution of artificial intelligence, automated access to information is now more than ever before. But the issue of the accuracy and reliability of responses generated is also a significant problem. In this chapter, the background of the problem to be discussed in this project is provided, along with the motivation behind the enhancement of the existing methods, the critical limitations of the current systems, and the purposes and scope of the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) framework.

1.1 Preamble

Large Language Models (LLM) have made systems much more efficient at responding to natural language inquiries. Such models, however, usually produce reactions using the only knowledge they possess internally which maybe out of date, incomplete or erroneous. In order to solve this problem, Retrieval-Augmented Generation (RAG) was proposed [2], in which a document of interest is firstly retrieved externally, and then generated answer is generated. Although RAG enhances factual grounding, a traditional RAG system uses primarily unstructured text retrieval and does not work well on complex multi-hop queries in need of integrating information across multiple sources of information [3]. This project presents an enhanced version of a knowledge graph combined with a neural retrieval and Generation (KG-RAG) architecture named [4] to enhance reasoning, accuracy and consistency of responses as introduced by the new framework: Knowledge Graph Enhanced RAG (KG-RAG).

1.2 Motivation

This project is driven by the fact that current question-answering systems are limited and that there is a growing need to know precise, reputable and explainable answers in educational and knowledge-based platforms of use, as well as the limitation posed by these systems and systems that provide such answers [5]. The key inspirational determinants include:

- **Low Precision in Traditional RAG Systems:** Conventional RAG frameworks retrieve large volumes of text, which often include irrelevant or weakly related information, thereby reducing the precision of generated answers [6].
- **Inability to Support Multi-Hop Reasoning:** Many real-world queries require combining information from multiple documents that are not directly similar. Standard retrieval methods are unable to effectively handle such multi-hop reasoning [7].
- **Limited Relationship Understanding:** Vector-based retrieval techniques fail to capture explicit relationships between entities across documents, resulting in incomplete or shallow reasoning [8].
- **Redundant and Noisy Supporting Evidence:** Retrieved context frequently contains duplicate or unnecessary supporting facts, which lowers interpretability and affects evaluation accuracy [9].
- **Need for Trustworthy and Explainable Outputs:** High-stakes domains such as education require answers that are not only correct but also well-supported and transparent [10].

These challenges motivate the integration of Knowledge Graphs with intelligent filtering mechanisms to improve retrieval quality, reduce noise, and generate concise, well-supported, and reliable answers [11].

1.3 Literature Study

RAG was proposed to improve the accuracy of language models by retrieving relevant documents before generating answers [2]. This approach helps reduce hallucination by grounding responses in external knowledge. However, traditional RAG systems rely mainly on unstructured text and semantic similarity, which limits their ability to answer complex queries that require information from multiple sources [12].

To overcome this limitation, researchers have explored the use of Knowledge Graphs (KGs) in question-answering systems [13]. Knowledge graphs represent information as entities and relationships, enabling structured and multi-hop reasoning [1]. Graph-guided retrieval methods improve the ability to connect related concepts across documents, but uncontrolled graph expansion often introduces irrelevant information, reducing precision [7].

Neural reranking techniques have been introduced to improve retrieval quality by better matching queries with relevant documents [14, 15]. While reranking improves relevance, most existing systems lack effective mechanisms to control noise and eliminate duplicate supporting evidence.

From the literature, it is clear that although knowledge graphs improve reasoning and neural reranking enhances relevance, existing approaches do not adequately balance recall, precision, and clean evidence selection. This project addresses these gaps by proposing a KG-RAG framework that combines graph-based expansion, neural reranking, and supporting-fact deduplication [16, 11].

1.4 Problem Definition

RAG models rely mainly on unstructured text retrieval, which limits reasoning accuracy, contextual understanding, and personalization, especially in educational applications [17]. This reduces the effectiveness of intelligent tutoring systems compared to human-led or more structured pedagogical approaches [5].

1.5 Objectives of the project

The main objective of this project is to design and evaluate a KG-RAG framework that improves the precision, reliability, and explainability of question-answering systems, particularly for multi-hop reasoning tasks. The specific objectives of the project are:

1. To implement a baseline Retrieval-Augmented Generation (RAG) system using dense vector retrieval [12] over a document corpus and a generative language model.
2. To construct or integrate a knowledge graph and develop mechanisms for entity linking, graph expansion, and knowledge graph querying to support relational reasoning [4, 8].
3. To design a multi-stage retrieval pipeline that combines broad vector retrieval, knowledge-graph-based expansion, and neural reranking [18, 15] to improve the precision of retrieved context.
4. To develop a supporting-fact extraction mechanism with deduplication and controlled selection of evidence to reduce redundancy and noise in retrieved information [9].
5. To evaluate the proposed KG-RAG framework on a multi-hop question-answering benchmark [3] using standard metrics such as Exact Match (EM), F1 score, Supporting Fact Precision, and Joint Precision.

Chapter 2

SOFTWARE REQUIREMENT SPECIFICATION

This chapter presents the Software Requirement Specification (SRS) for the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system. It describes the required system functionalities, performance characteristics, and operational constraints that guide the design and development of the system. This chapter serves as a foundation for subsequent design and implementation chapters by clearly defining what the system is intended to accomplish.

2.1 Overview of SRS

The Software Requirement Specification (SRS) is a formal document that describes the expected behavior, capabilities, and quality attributes of a software system. It identifies the system requirements in a structured manner so that developers, evaluators, and users share a common understanding of system objectives. In this project, the SRS specifies the functional features, non-functional constraints, operating environment, and assumptions of the KG-RAG framework, providing a reference for design, implementation, and evaluation.

2.2 Requirement Specifications

Functional requirements specify the core operations and services that the proposed system must perform to achieve its intended purpose. In the context of the Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) framework, functional requirements define how the system processes user queries, retrieves relevant information from a document corpus, expands results using knowledge graph relationships [4], filters the retrieved content through neural reranking [14], and generates final answers based only on the selected context. These requirements also include the extraction, deduplication, and presentation of supporting facts [9], as well as the evaluation of system performance using standard metrics. Functional requirements therefore describe what the system must do to ensure accurate, reliable, and explainable question answering [1].

2.2.1 Functional Requirements

Functional requirements describe the specific operations and services that the proposed system must perform to achieve its intended objectives. For the Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) framework, these requirements define how the system processes queries, retrieves and filters information, and generates evidence-based answers. The major functional requirements of the system are as follows:

- **FR1: Query Processing**

The system shall accept user queries in natural language and initiate the retrieval process for answering the query [19].

- **FR2: Document Retrieval**

The system shall retrieve an initial set of relevant documents from the document corpus using dense vector-based semantic similarity [12].

- **FR3: Knowledge Graph Expansion**

The system shall expand the retrieved results by identifying related entities and relationships from the knowledge graph to support multi-hop reasoning [7, 8].

- **FR4: Neural Reranking and Filtering**

The system shall apply neural reranking [15] to the expanded results and filter the retrieved content to retain only the most relevant information for answer generation.

- **FR5: Answer Generation and Supporting Fact Extraction**

The system shall generate the final answer using only the selected context [6] and extract supporting facts by removing duplicate and redundant evidence before presenting the output [9].

2.2.2 Use case diagram

The use case diagram represents the functional behavior of the proposed KG-RAG system and illustrates the interactions between the user and the system. It provides a high-level view of the major operations involved in query processing, information retrieval, knowledge graph expansion, reranking, and answer generation. The diagram helps in understanding the system scope by clearly defining the responsibilities of the user and the sequence of system functionalities required to produce accurate and well-supported responses.

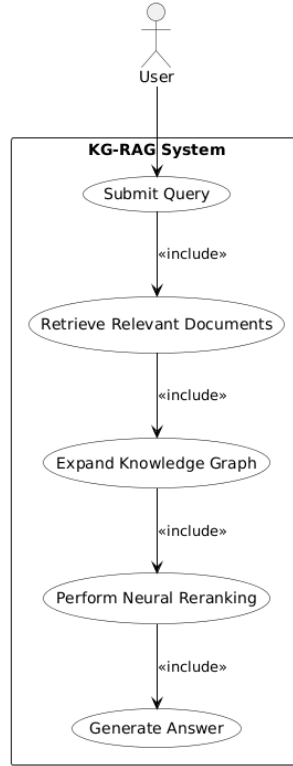


Figure 2.1: Use Case Diagram of the Proposed KG-RAG System

Based on the functional requirements and the use case diagram, five major use cases have been identified for the proposed KG-RAG system. These use cases collectively represent the complete workflow of query processing, knowledge retrieval, reasoning, and answer generation. Each use case captures a distinct functional responsibility of the system and is described briefly below.

- **Submit Query:** This use case allows the user to provide a natural language query to the system, initiating the question-answering process.
- **Retrieve Relevant Documents:** In this use case, the system retrieves semantically relevant documents from the document corpus using dense vector-based similarity [18].
- **Expand Knowledge Graph:** This use case enables the system to enhance the retrieved information by exploring related entities and relationships from the knowledge graph to support multi-hop reasoning [7].
- **Perform Neural Reranking:** The system applies neural reranking techniques to filter and prioritize the most relevant information from the expanded results [14].
- **Generate Answer:** In this final use case, the system synthesizes a grounded answer using the selected context and presents the result to the user [2].

2.2.3 Use Case descriptions using scenarios, strictly as per Pressman Template

Use Case ID	UC-1
Use Case Name	Submit Query and Generate Answer
Primary Actor	User
Description	This use case describes how a user submits a natural language query to the KG-RAG system and receives a grounded answer based on retrieved and processed knowledge.
Preconditions	• The system is operational. • The document corpus and knowledge graph are available and indexed.
Postconditions	• A final answer is generated and displayed to the user. • Supporting information is internally processed for answer generation.
Main Success Scenario	1. The user submits a query to the system. 2. The system retrieves relevant documents from the document corpus. 3. The system expands the retrieved results using knowledge graph relationships. 4. The system performs neural reranking to filter the most relevant information. 5. The system generates a grounded answer using the selected context. 6. The final answer is displayed to the user.
Alternate Flow	• If no relevant documents are retrieved, the system informs the user that no sufficient information is available to generate an answer.

Table 2.1: Use Case Description for UC-1: Submit Query and Generate Answer

Use Case ID	UC-2
Use Case Name	Knowledge Graph Expansion for Multi-hop Reasoning
Primary Actor	User
Description	This use case represents the system's internal process of expanding retrieved information using the knowledge graph to support multi-hop reasoning.
Preconditions	• The user has submitted a valid query. • Initial relevant documents have been retrieved.
Postconditions	• Additional related entities and relationships are identified from the knowledge graph. • The expanded information is passed to the reranking stage.
Main Success Scenario	1. The system identifies entities from the initially retrieved documents. 2. The system queries the knowledge graph to obtain related entities and relationships. 3. The system merges the expanded knowledge with the retrieved documents. 4. The combined information is forwarded for neural reranking.
Alternate Flow	• If no related entities are found in the knowledge graph, the system continues with the originally retrieved documents.

Table 2.2: Use Case Description for UC-2: Knowledge Graph Expansion for Multi-hop Reasoning

Use Case ID	UC-3
Use Case Name	Neural Reranking and Filtering
Primary Actor	User
Description	This use case describes how the system ranks and filters the expanded information to retain only the most relevant content for answer generation.
Preconditions	• Knowledge graph expansion has been completed. • Expanded documents and entities are available.
Postconditions	• Irrelevant and redundant information is removed. • A refined set of highly relevant evidence is produced.
Main Success Scenario	1. The system receives expanded information from the knowledge graph. 2. The system applies neural ranking models to score each piece of evidence. 3. Low-relevance and duplicate items are filtered out. 4. The refined information set is prepared for answer generation.
Alternate Flow	• If reranking fails or insufficient high-quality evidence is available, the system proceeds with the best available filtered results.

Table 2.3: Use Case Description for UC-3: Neural Reranking and Filtering

Use Case ID	UC-4
Use Case Name	Answer Generation
Primary Actor	User
Description	This use case describes how the system generates a grounded answer using the selected and filtered context obtained from previous processing stages.
Preconditions	• Relevant and filtered evidence is available. • Neural reranking has been completed successfully.
Postconditions	• A final answer is generated based on the selected context. • The answer is prepared for presentation to the user.
Main Success Scenario	1. The system receives the filtered context from the reranking stage. 2. The language model processes the context to generate an answer. 3. The system ensures the response is grounded in the provided evidence. 4. The final answer is formatted for display.
Alternate Flow	• If the model cannot generate a confident answer, the system returns a message indicating insufficient information.

Table 2.4: Use Case Description for UC-4: Answer Generation

Use Case ID	UC-5
Use Case Name	Supporting Fact Extraction and Presentation
Primary Actor	User
Description	This use case represents how the system extracts supporting facts from the selected evidence and presents them along with the generated answer to ensure transparency and explainability.
Preconditions	• A final answer has been generated. • Supporting evidence is available from previous stages.
Postconditions	• Supporting facts are extracted and deduplicated. • The answer and evidence are displayed to the user.
Main Success Scenario	1. The system identifies relevant evidence used in answer generation. 2. Duplicate and redundant facts are removed. 3. The system associates each fact with the generated answer. 4. The final output is presented with supporting information.
Alternate Flow	• If supporting facts cannot be clearly extracted, the system displays the answer without explicit evidence annotations.

Table 2.5: Use Case Description for UC-5: Supporting Fact Extraction and Presentation

2.2.4 Nonfunctional Requirements

Nonfunctional requirements define the quality attributes and constraints under which the proposed KG-RAG system must operate. These requirements focus on performance, reliability, usability, security, and scalability to ensure that the system produces accurate, explainable, and efficient responses suitable for knowledge-intensive applications [17].

Performance Requirements

The system shall retrieve relevant documents, expand knowledge graph entities, perform neural reranking, and generate a response within an acceptable response time for interactive use. The reranking and filtering stages must efficiently handle candidate sets without significantly increasing latency [15].

Reliability Requirements

The system shall consistently produce grounded answers based on retrieved evidence. In cases where sufficient information is unavailable, the system must return a clear and informative message rather than generating unsupported or hallucinated responses [2].

Usability Requirements

The system shall provide a simple and intuitive interface that allows users to submit natural language queries and view answers along with supporting evidence. Output responses must be readable, well-structured, and easy to interpret by non-technical users [10].

Security Requirements

The system shall restrict access to the document corpus, vector database, and knowledge graph to authorized components only. User queries and retrieved content shall be processed securely without exposing internal data or model configurations.

Scalability Requirements

The system shall support expansion of the document corpus and knowledge graph without major architectural changes. The retrieval and reranking components must be capable of handling increasing data volume while maintaining retrieval quality [4].

2.2.5 Software and Hardware Requirement Specifications

This section describes the software tools, frameworks, and hardware resources required to implement and execute the proposed KG-RAG system effectively.

Software Requirements:

The following are the software requirements we have used in this system :

- **Operating System:** Windows, Linux, or macOS
- **Programming Language:** Python
- **Machine Learning Frameworks:** PyTorch / HuggingFace Transformers [20]
- **Language Model:** LLaMA-based or equivalent large language model
- **Vector Database:** FAISS / Chroma / equivalent dense retrieval engine [18]
- **Knowledge Graph Tools:** Neo4j / RDF-based graph framework or custom KG storage [21]
- **Development Tools:** VS Code / Jupyter Notebook
- **Visualization Tools:** Graph visualization libraries (if applicable)

Hardware Requirements:

The following are the hardware requirements we have used in this system

- **Processor:** Minimum Intel i5 / AMD Ryzen 5 or equivalent
- **Memory (RAM):** Minimum 8 GB (16 GB recommended for model inference and indexing)
- **Storage:** At least 20 GB free space for document corpus, embeddings, and knowledge graph
- **GPU (Optional):** NVIDIA GPU with CUDA support for faster neural reranking and language model inference

Chapter 3

PROPOSED SYSTEM

This chapter presents the design and working of the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system. It explains the overall system architecture, key components, and the methodology used to integrate document retrieval, knowledge graph reasoning, and neural reranking for answer generation. The chapter describes how user queries are processed through successive stages of retrieval, knowledge expansion, filtering, and generation to produce accurate, grounded, and explainable responses [4].

3.1 Description of Proposed System.

The proposed system implements a Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) framework to improve the accuracy and reliability of question answering. Unlike traditional RAG systems that rely only on unstructured text retrieval [2], this system integrates structured knowledge from a knowledge graph with neural retrieval and language model generation [11].

When a user submits a query, relevant documents are first retrieved using dense semantic search [12]. The retrieved information is then expanded through the knowledge graph to capture related entities and relationships, enabling multi-hop reasoning [7]. A neural reranking module filters and prioritizes the most relevant content [14], which is finally provided to the language model for answer generation based only on the selected evidence [6]. Supporting facts are retained to ensure transparency [9].

Through this pipeline, the system produces concise, grounded, and explainable answers, particularly for complex queries [16].

3.1.1 Description of Target Users

The proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system is intended for users who require accurate, explainable, and context-aware answers from large document collections. The system is designed to support knowledge-intensive queries that involve multi-hop reasoning across multiple sources [3].

The primary target users include:

- **Students and Learners:** Users seeking reliable and well-supported answers for academic and conceptual queries in educational domains [10].
- **Researchers and Analysts:** Users who require precise retrieval of relevant information and the ability to trace answers to structured knowledge and supporting evidence [1].
- **Educators and Content Developers:** Users who need explainable outputs with verified sources for creating learning materials and assessments [5].
- **Technical Users / Developers:** Users interested in experimenting with advanced retrieval and reasoning pipelines that combine vector search, knowledge graphs, and large language models [8].

The system is designed to be **domain-adaptable** [22], enabling deployment in specialized knowledge areas by integrating domain-specific document corpora and knowledge graphs, while maintaining usability through natural language interaction.

3.1.2 Advantages / Applications of Proposed System

The proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system offers significant improvements over traditional retrieval-based question-answering systems. By integrating structured knowledge graphs with neural retrieval and generation models, the system enhances accuracy, interpretability, and reasoning across multiple information sources [4, 11]. This section highlights the key advantages of the system and its major application areas.

Advantages of the Proposed System:

The proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system offers several technical and functional advantages over traditional RAG-based approaches. By integrating structured knowledge graphs with neural retrieval and generation, the system improves reasoning capability, answer reliability, and interpretability. The key advantages are summarized below.

- **Improved Answer Accuracy:** Combines document retrieval with structured knowledge to reduce hallucinations and irrelevant responses [2].
- **Multi-hop Reasoning Capability:** Enables reasoning across multiple documents and related entities using knowledge graph relationships [7].
- **Explainability:** Provides supporting facts and traceable evidence for generated answers [9].

- **Noise Reduction:** Filters redundant and weakly related content using neural reranking and KG-based expansion [15].
- **Domain Adaptability:** Can be customized for specific domains by integrating domain-specific corpora and knowledge graphs [21].

Applications of the Proposed System:

Due to its ability to perform multi-hop reasoning, filter noisy information, and generate evidence-supported responses, the proposed KG-RAG framework is suitable for a wide range of knowledge-intensive applications. The major application areas of the system are listed below.

- **Educational Question Answering Systems:** For students and instructors requiring reliable and explainable academic responses [17].
- **Research Assistance Tools:** Supports literature analysis, entity-based exploration, and cross-document reasoning [1].
- **Intelligent Tutoring Systems:** Enhances personalized learning through context-aware, knowledge-grounded answers [5].
- **Enterprise Knowledge Management:** Enables accurate retrieval from organizational knowledge bases [8].
- **Decision Support Systems:** Assists in domains requiring fact-based reasoning such as healthcare, law, and technical support [22].

3.1.3 Scope (Boundary of Proposed System)

The scope of the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system is limited to processing user queries over a predefined document corpus and an associated knowledge graph. The system performs semantic retrieval, knowledge graph expansion, neural reranking, and answer generation using a large language model [16].

The system focuses on providing accurate, explainable, and evidence-supported responses for knowledge-intensive queries. It does not include real-time web search, autonomous knowledge graph construction, or user behavior analytics. External data sources, system training, and model fine-tuning are considered outside the scope of this implementation.

Thus, the proposed system is confined to controlled input data, structured knowledge integration, and answer synthesis within the defined domain.

Chapter 4

SYSTEM DESIGN

This chapter presents the system design of the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) framework. It describes the architecture and internal organization of the system, explaining how user queries are processed through document retrieval, knowledge graph expansion, neural reranking, and answer generation [4]. Design representations such as architecture diagrams, class diagrams, and sequence diagrams are used to illustrate component interactions and data flow. This chapter translates the functional requirements into a clear and implementable system design.

4.1 Architecture of the System

The proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system follows a layered architecture that integrates neural document retrieval with structured knowledge graph reasoning to generate accurate, explainable answers [11]. Each layer performs a well-defined function in the overall query processing pipeline, as shown in Figure 4.1.

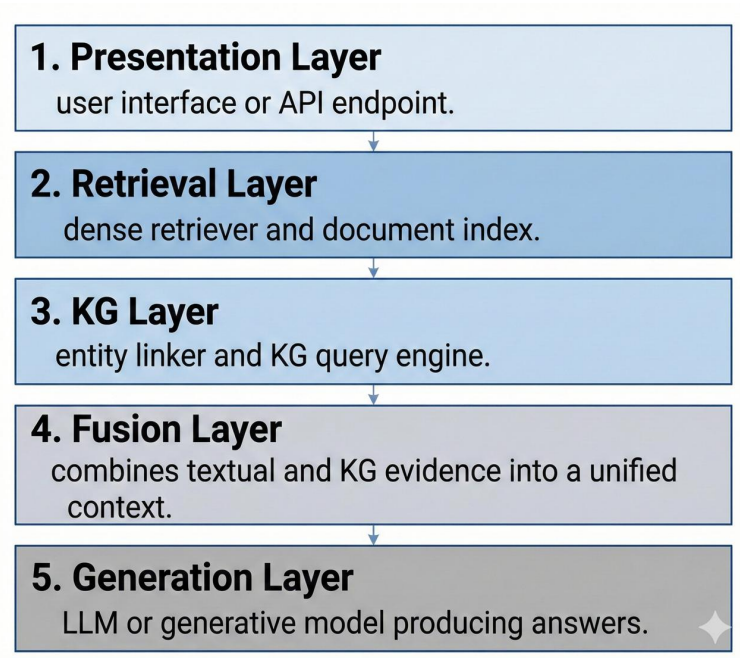


Figure 4.1: Layered Architecture of the Proposed KG-RAG System

The responsibilities of each layer are described below:

1. Presentation Layer

The Presentation Layer provides the user interface or API through which natural language queries are submitted. It acts as the interaction point between the user and the system, forwarding requests to the backend and displaying the generated answers along with supporting evidence [19].

2. Retrieval Layer

The Retrieval Layer performs dense semantic search over the document corpus using vector embeddings [18]. It retrieves the top-K most relevant documents based on similarity to the input query and serves as the initial filtering stage to reduce the search space [12].

3. Knowledge Graph (KG) Layer

The Knowledge Graph Layer identifies key entities from the retrieved documents using entity linking techniques. It queries the knowledge graph to obtain related entities and relationships, enabling multi-hop reasoning across multiple sources [7, 8].

4. Fusion Layer

The Fusion Layer combines textual evidence from retrieved documents with structured knowledge obtained from the knowledge graph. It applies filtering and neural reranking [14, 15] to remove redundant or weakly related information and constructs a unified context for answer generation.

5. Generation Layer

The Generation Layer employs a Large Language Model (LLM) to generate the final answer using only the selected and validated context [6]. It ensures that responses are grounded in retrieved evidence and presents both the answer and relevant supporting facts for explainability [9].

4.2 Class Diagram

A class diagram is a Unified Modeling Language (UML) representation that models the static structure of a system by depicting its classes, attributes, methods, and the relationships among them. It provides a clear view of system organization and supports implementation by defining component responsibilities and interactions.

Description of the KG-RAG Class Diagram

The class diagram of the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system illustrates the major software components and their interactions, along with the multiplicities that define how objects are associated.

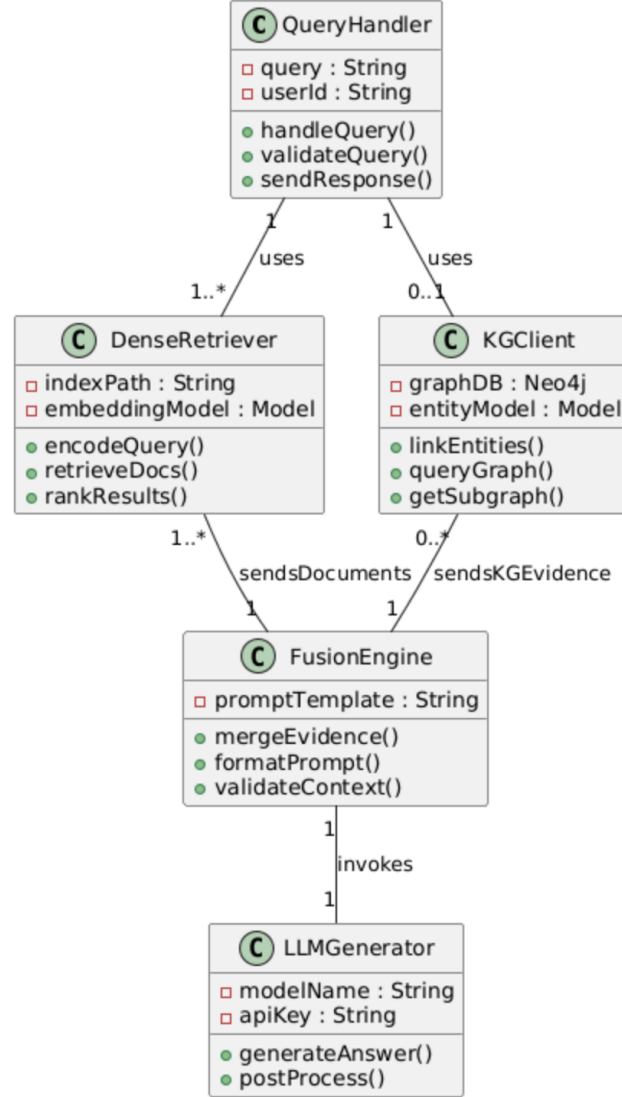


Figure 4.2: UML Class Diagram of the Proposed KG-RAG System

- **QueryHandler (Controller Class):** This class manages user requests by validating queries, coordinating document retrieval, and sending responses. The $1..*$ association between **QueryHandler** and **DenseRetriever** indicates that one query handler may utilize multiple retrieval operations for a single request [2]. The $0..1$ relationship with **KGClient** shows that access to the knowledge graph is optional and depends on the query context.

- **DenseRetriever (Document Retrieval Module):** This class encodes user queries and retrieves relevant documents from the vector index [12]. The $1..* to 1$ relationship between **DenseRetriever** and **FusionEngine** indicates that multiple retrieved documents are forwarded to a single fusion component for integration.
- **KGClient (Knowledge Graph Interface):** This component performs entity linking and graph queries to obtain structured knowledge [4]. The $0..* to 1$ association with **FusionEngine** signifies that zero or more pieces of graph-based evidence may be provided to the fusion module depending on availability and relevance.
- **FusionEngine (Evidence Integration Layer):** This class merges textual evidence and knowledge graph information into a unified context [16]. The $1 to 1$ relationship with **LLMGenerator** indicates that each fused context is passed to exactly one language model instance for answer generation.
- **LLMGenerator (Answer Generation Module):** This class generates the final grounded answer based on the processed context and performs post-processing [6]. The $1 to 1$ invocation from the **FusionEngine** ensures a single, controlled generation step per query.

4.3 Sequence diagram

A sequence diagram is a UML behavioral diagram that illustrates how objects or components interact with each other over time to accomplish a specific functionality. It shows the order of message exchanges between different system modules during the execution of a use case.

The sequence diagram illustrates the step-by-step interaction between system components during query processing in the proposed KG-RAG framework. It shows how a user request flows through retrieval, knowledge expansion, fusion, and answer generation.

- The user submits a query through the **QueryHandler**, which validates the input.
- The **DenseRetriever** retrieves the top-K relevant documents using semantic search [12].
- The **KGClient** expands the retrieved information by performing entity linking and querying the knowledge graph [1].
- The **FusionEngine** merges textual and KG evidence, filters noise, and builds the final context [9].
- The **LLMGenerator** generates a grounded answer from the fused context [2].

- The final answer is returned and displayed to the user.

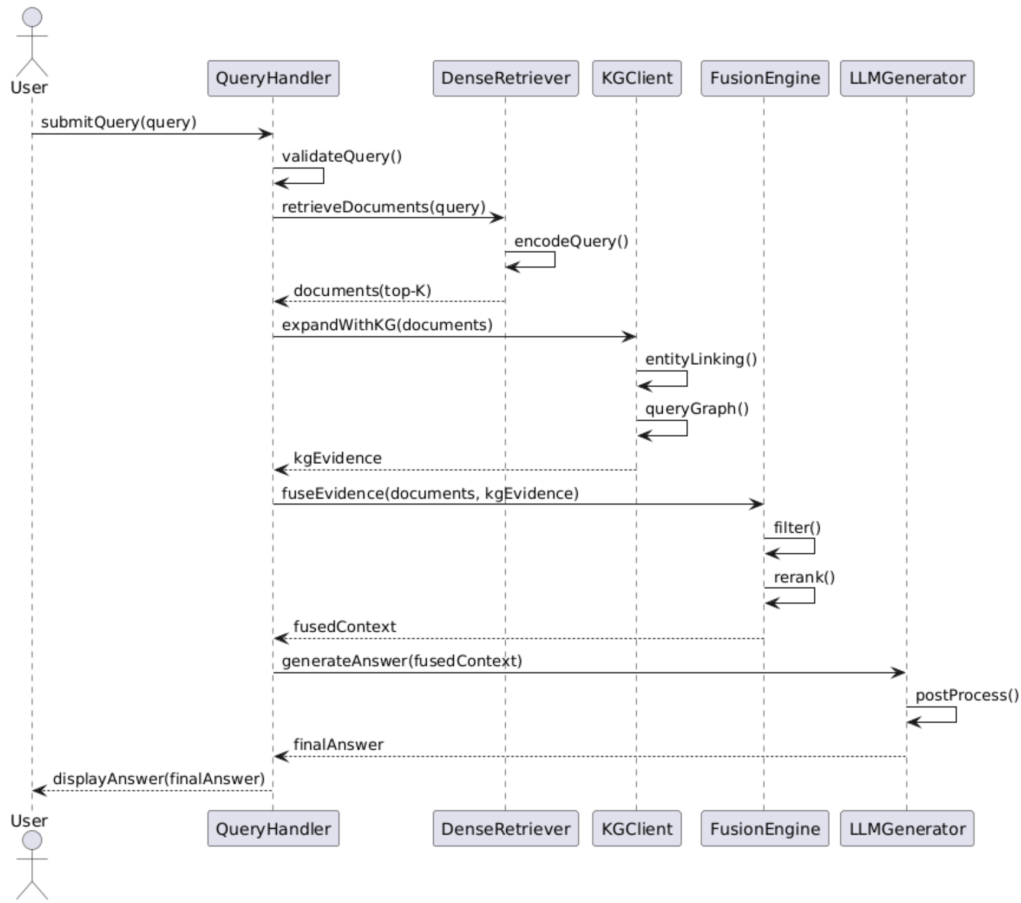


Figure 4.3: Sequence Diagram of the Proposed KG-RAG System

4.4 ER diagram and schema

An Entity–Relationship (ER) diagram represents the logical data structure of the system by identifying key entities, their attributes, and the relationships among them. It provides a clear view of how data is organized and how different components interact at the database level.

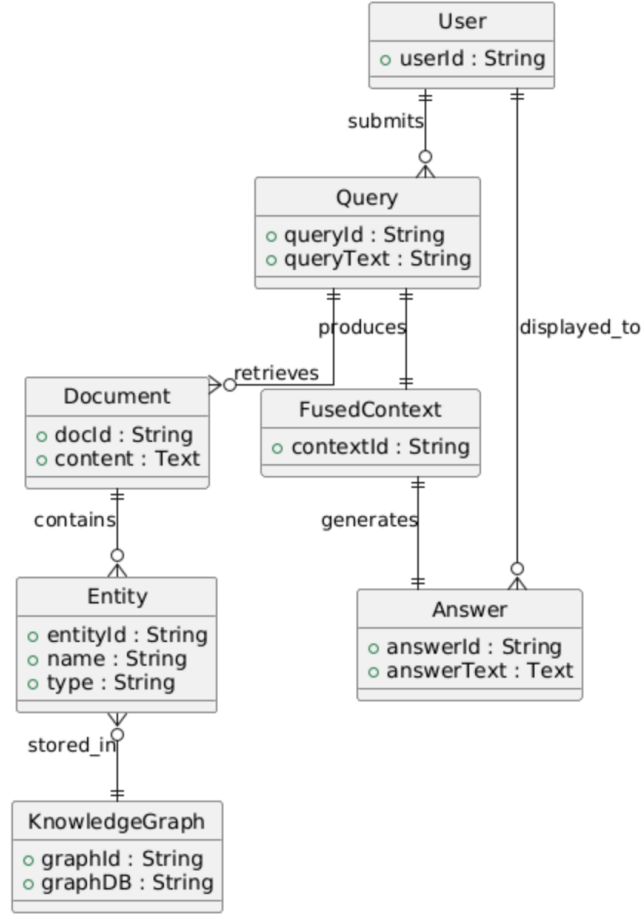


Figure 4.4: ER Diagram of the Proposed KG-RAG System

- **User:** Represents the end user of the system. A user submits one or more queries, and the final answers are displayed to the user.
- **Query:** Stores the natural language input provided by the user. Each query is associated with a single user and produces a fused context after processing.
- **Document:** Represents textual sources retrieved from the document corpus [12]. A query retrieves one or more documents that contain relevant information.
- **Entity:** Denotes important concepts extracted from documents. Each document contains multiple entities that are used for knowledge linking [4].
- **KnowledgeGraph:** Stores entities and their relationships in a structured form. Each entity is stored in the knowledge graph to enable semantic expansion and multi-hop reasoning [7].
- **FusedContext:** Combines relevant document content and knowledge graph evidence. It acts as an intermediate representation that generates the final answer.

- **Answer:** Represents the system output. Each fused context produces one answer, which is then displayed to the user.

4.5 State transition diagram

A state transition diagram represents the dynamic behavior of a system by showing how it moves between different operational states in response to events. It helps in understanding the control flow, decision points, and error handling mechanisms of the system during execution.

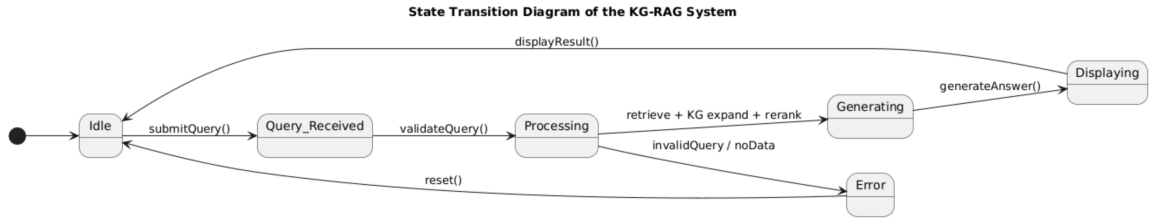


Figure 4.5: State Transition Diagram of the Proposed KG-RAG System

- **Idle**

This is the initial state where the system is waiting for user input. No processing occurs until a new query is submitted by the user.

- **Query_Received**

The system accepts the natural language query from the user interface or API. The query is captured and prepared for validation before further processing.

- **Processing**

In this state, the system performs document retrieval, knowledge graph expansion, and neural reranking [14].

Relevant documents and KG evidence are gathered and filtered to build a high-quality context.

- **Generating**

The fused context is passed to the language model to generate a grounded and accurate answer [2].

Answer generation is strictly based on retrieved evidence to minimize hallucinations.

- **Displaying**

The final answer and supporting facts are presented to the user [9].

After displaying the result, the system returns to the Idle state for the next query.

- **Error**

This state is triggered when the query is invalid or when no sufficient data is found.

The system safely reports the issue and resets without interrupting future operations.

4.6 Data Set Description(if applicable)

The proposed KG-RAG system uses a combination of unstructured text data, structured knowledge, and evaluation samples to support retrieval, reasoning, and answer generation. The dataset is organized into three major components: the document corpus, the knowledge graph, and the evaluation set [3].

- **Document Corpus:** Contains textual documents that are embedded and indexed for dense semantic retrieval [18]. It serves as the primary information source for answering user queries.
- **Knowledge Graph:** Stores entities and their relationships in a structured form, enabling entity linking and multi-hop reasoning [1]. It enriches the retrieved documents with contextual and relational knowledge.
- **Evaluation Set:** Consists of annotated question–answer pairs used to assess system performance [3]. Metrics such as Exact Match (EM), F1-score, and hallucination rate are applied to evaluate accuracy, relevance, and grounding.

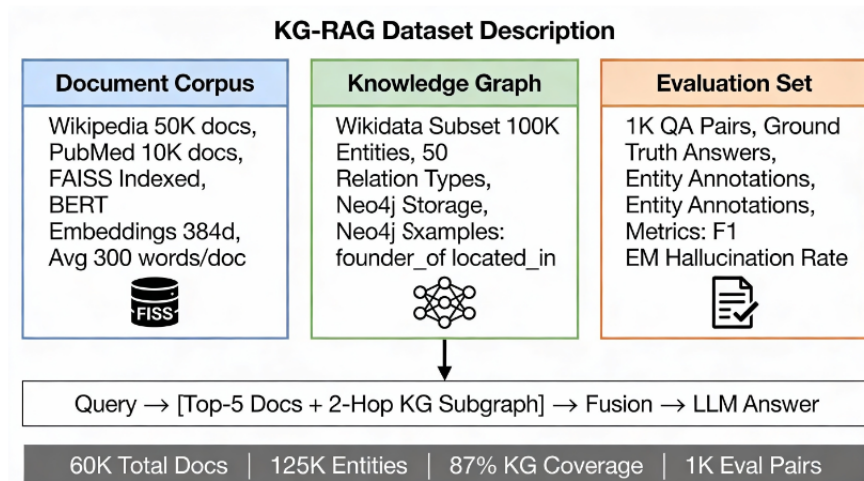


Figure 4.6: Dataset Description: Document Corpus and Knowledge Graph

Chapter 5

IMPLEMENTATION

This chapter presents the implementation of the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system. It explains how the system components are designed, developed, and integrated to process user queries, retrieve relevant documents, expand knowledge using a graph-based structure, and generate accurate, evidence-grounded answers [4]. The chapter details the methodology, algorithms, and module-wise implementation, thereby providing a clear view of the internal working and execution flow of the system.

5.1 Proposed Methodology

This section describes the methodology adopted for implementing the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system. The approach integrates semantic document retrieval [12] with structured knowledge graph reasoning [1] to generate accurate, explainable, and context-grounded answers. Fig. 5.1 illustrates the complete workflow of the proposed system.

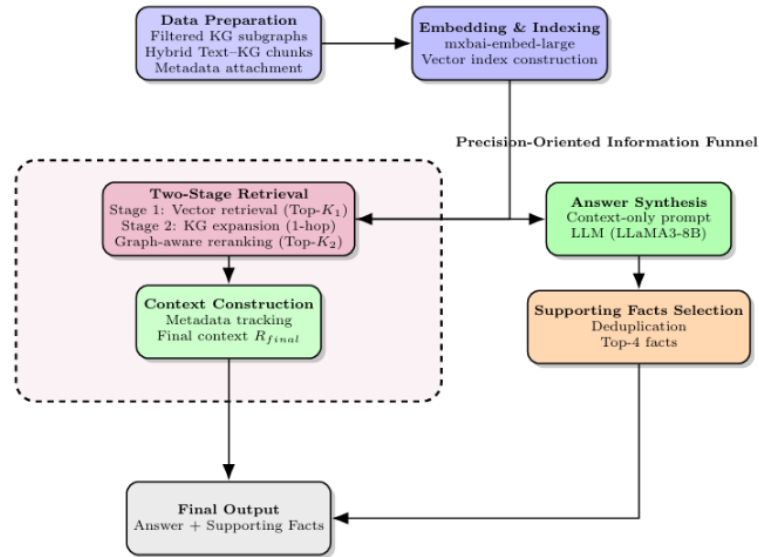


Figure 5.1: Workflow of the Proposed KG-RAG Methodology

- **Data Preparation and Indexing**

Relevant documents and knowledge graph subgraphs are filtered and preprocessed.

Hybrid text–KG chunks are created and enriched with metadata for better traceability [8].

Dense embeddings are generated using Sentence-BERT [18] and stored in a vector index to support efficient semantic retrieval.

- **Two-Stage Retrieval Process**

In the first stage, the system retrieves the top-K most relevant documents using vector similarity search [12].

In the second stage, the retrieved results are expanded using the knowledge graph by identifying related entities [7].

This enables multi-hop reasoning and graph-aware reranking by combining structured and unstructured knowledge [11].

- **Context Construction and Fusion**

Textual evidence from documents and structured information from the knowledge graph are merged [16].

Redundant, noisy, and weakly relevant content is filtered to improve context quality using late interaction models like ColBERT [15].

A unified and optimized context is constructed to ensure accurate and reliable answer generation.

- **Answer Synthesis using LLM**

The fused context is provided to a Large Language Model for answer generation [2].

The model is restricted to the selected evidence to reduce hallucination and maintain factual grounding [6].

This ensures that responses are coherent, precise, and aligned with retrieved knowledge.

- **Supporting Facts Selection and Output**

The system identifies and selects the most relevant supporting facts used in generating the answer [9].

Both the final response and its evidence are presented to the user for transparency.

This improves explainability and makes the system suitable for academic, research, and enterprise applications [17].

5.2 Description of Modules

This section describes the major functional modules of the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system. Each module is defined in terms

of its input, output, and internal working. The modular design improves clarity, scalability, and ease of implementation while ensuring a structured flow of information from query submission to answer generation.

Module 1: Query Handling Module

This module acts as the entry point of the system by receiving and validating user queries before further processing.

Input: User query (natural language text)

Output: Validated query

Description:

This module receives the user's natural language query through the interface or API endpoint. It performs basic validation such as checking for empty or malformed inputs [19]. Only validated queries are forwarded to the retrieval module, thereby preventing unnecessary system overhead.

Module 2: Document Retrieval Module

This module retrieves the most relevant documents for a validated query using dense semantic similarity search.

Input: Validated user query

Output: Top-K relevant documents

Description:

This module performs dense semantic search over the indexed document corpus. The input query is encoded into vector embeddings and compared against document embeddings using similarity measures [18]. The module retrieves the top-K most relevant documents [12], which serve as the initial evidence set for downstream reasoning and answer generation.

Module 3: Knowledge Graph Expansion Module

This module enriches retrieved documents by extracting related entities and relationships from the knowledge graph.

Input: Retrieved documents

Output: Related entities and relationships (KG evidence)

Description:

This module enhances the retrieved textual evidence by identifying key entities using entity linking techniques [4]. It queries the knowledge graph to extract connected entities and relations, thereby supporting multi-hop reasoning [7]. The expanded knowledge provides structured context that complements unstructured document content [8].

Module 4: Fusion and Reranking Module

This module combines textual evidence and knowledge graph information and ranks them to form a high-quality context.

Input: Retrieved documents and KG evidence

Output: Fused and ranked context

Description:

This module integrates textual information from documents with structured knowledge obtained from the knowledge graph [11]. Redundant and weakly related information is filtered out using neural reranking techniques [14, 15]. The resulting fused context represents a high-quality and concise evidence set that is optimized for accurate answer generation [9].

Module 5: Answer Generation Module

This module generates the final grounded answer using a large language model based on the fused contextual evidence.

Input: Fused context

Output: Final answer with supporting facts

Description:

This module uses a Large Language Model (LLM) to generate the final response based solely on the selected fused context [6]. By grounding the output in retrieved and structured evidence [2], the system minimizes hallucinations and improves reliability. Supporting facts are also extracted and presented to ensure transparency and explainability of the generated answer [9].

Chapter 6

TESTING

This chapter presents the testing strategy adopted for validating the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system. It describes the test plan, types of testing performed, and representative test cases used to verify functional correctness, system integration, and output reliability. The testing process ensures that each module of the system operates as intended and that the overall pipeline produces accurate, grounded, and explainable responses [4, 11].

6.1 Test Plan and Test Cases

The testing of the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system was conducted to validate the correctness of query processing, document retrieval, knowledge graph expansion, evidence fusion, and answer generation. Both module-level and system-level tests were performed to ensure functional accuracy, robustness, and consistency with the project objectives [16].

6.1.1 Types of Testing Performed :

There are various types of testing performed on this model. Following are the types of testing:

Unit Testing

Unit testing was performed to verify the correctness of individual modules. Each component was tested independently before system integration.

- Query validation for correct input format [19].
- Dense retriever for accurate top-K document retrieval [12].
- KG client for entity linking and subgraph extraction [1].
- Fusion engine for proper evidence merging [8].
- LLM generator for response formatting [2].

Integration Testing

Integration testing ensured correct interaction among modules in the KG-RAG pipeline and verified smooth data flow across all stages [4].

- Retrieved documents passed correctly to KG expansion [7].
- KG evidence merged properly with textual data [11].
- Unified context provided to the LLM for answer generation [6].

System Testing

System testing evaluated end-to-end functionality using real user queries to validate accuracy, grounding, and stability.

- Correctness of final answers against multi-hop benchmarks [3].
- Availability of supporting facts and evidence traceability [9].
- Handling of invalid or empty queries to prevent hallucination [2].

Acceptance Testing

Acceptance testing confirmed that the system met the functional requirements and project objectives for intelligent tutoring and knowledge retrieval [17, 5].

- Support for multi-hop reasoning using the knowledge graph [7].
- Reduction of hallucinated responses through grounded generation [6].
- Delivery of evidence-backed answers for educational reliability [10].

6.1.2 Sample Test Cases

The following sample test cases illustrate the functional validation of key components in the proposed KG-RAG system. Each test case is designed to verify correct behavior under realistic usage scenarios, including scope validation, document retrieval, knowledge graph expansion, and grounded answer generation.

These test cases also ensure that the system responds only to queries that are relevant to the constructed dataset and knowledge graph. Queries outside the defined knowledge scope are intentionally rejected, thereby preventing unsupported or hallucinated responses and maintaining system reliability.

Test Case 1: Valid Query Submission

Field	Description
Test Case ID	TC-01
Objective	Verify that the system accepts only valid user queries that are within the scope of the constructed dataset and knowledge graph.
Input	“Who founded Tesla and where was it founded?”
Expected Output	Query is validated as in-scope and forwarded to the retrieval module; out-of-context queries are rejected with an appropriate response.
Result	Passed

Test Case 2: Document Retrieval

Field	Description
Test Case ID	TC-02
Objective	Validate top-K document retrieval for queries that are relevant to the dataset and indexed document corpus.
Input	Valid in-scope user query
Expected Output	Relevant documents are retrieved using dense embeddings; no documents are returned for out-of-scope queries.
Result	Passed

Test Case 3: Knowledge Graph Expansion

Field	Description
Test Case ID	TC-03
Objective	Verify entity linking and KG subgraph extraction for entities present in the constructed knowledge graph.
Input	Retrieved documents from an in-scope query
Expected Output	Related entities and relationships are identified; KG expansion is skipped if no relevant entities exist in the knowledge graph.
Result	Passed

Test Case 4: Answer Generation

Field	Description
Test Case ID	TC-04
Objective	Validate grounded answer generation only for queries within the dataset and knowledge graph scope.
Input	Fused textual and KG context for an in-scope query
Expected Output	Correct answer is generated with supporting evidence; if the query is outside the dataset/KG context, the system responds that it cannot answer the question.
Result	Passed

Chapter 7

RESULTS DISCUSSIONS

This chapter presents the experimental results obtained from evaluating the proposed Knowledge Graph Enhanced Retrieval-Augmented Generation (KG-RAG) system. The performance of the system is analyzed using standard evaluation metrics and compared with a baseline RAG model [12]. The results demonstrate the effectiveness of knowledge graph integration in improving answer accuracy, reducing hallucinations, and enabling explainable reasoning [11, 4].

7.1 Evaluation Overview

The system was evaluated on a multi-hop question answering benchmark, similar to HotpotQA or MuSiQue [3], using metrics such as Exact Match (EM), F1 score, Precision, Supporting Fact (SP) accuracy, and Joint metrics. These measures collectively assess not only whether the final answer is correct, but also whether the system retrieves and uses appropriate evidence to justify the answer [9, 16]. Three key result tables are analyzed: **Answer Quality**, **Supporting Fact Metrics**, and **Joint Metrics**.

7.2 Answer Quality Analysis

This table compares the overall answer quality of the baseline RAG system and the proposed KG-RAG model. It evaluates how accurately the system produces final answers using standard metrics such as Exact Match (EM), F1 score, and Precision [18]. The results indicate consistent improvement when structured knowledge from the knowledge graph is incorporated [7]. This demonstrates that KG-RAG enhances both correctness and reliability of generated responses by grounding the LLM in verifiable graph subgraphs [6].

Table 7.1: Answer Quality Comparison

Metric	Baseline RAG	Proposed KG-RAG
Exact Match (EM)	0.4676	0.4784 (+2.31%)
F1 Score	0.6029	0.6085 (+0.93%)
Precision	0.6264	0.6348 (+1.34%)

7.3 Supporting Fact and Explainability Analysis

This table evaluates how effectively the system retrieves and uses supporting facts for answer generation [9]. The metrics measure the accuracy, precision, and recall of the evidence used to justify answers. The significant improvements in Supporting Fact Precision and F1 show that KG-RAG produces more grounded and explainable responses [17]. This confirms that the system reduces unsupported or hallucinated content by utilizing the explicit relational structure of the knowledge graph [8].

Table 7.2: Supporting Fact Metrics

Metric	Baseline	KG-RAG	Change
SP EM	0.0000	0.0027	+0.0027
SP F1	0.3381	0.4991	+47.61%
SP Precision	0.2173	0.4052	+86.48%
SP Recall	0.8202	0.6756	-17.63%

7.4 Joint Metrics (Multi-hop Reasoning Performance)

This table presents joint evaluation metrics that assess the system’s ability to answer multi-hop questions requiring reasoning across multiple pieces of evidence [7]. The Joint F1 score increases by **54.50%**, while Joint Precision improves by **96.18%**, demonstrating a significant enhancement in reasoning accuracy [11]. These results indicate that the proposed KG-RAG framework effectively integrates unstructured text with structured knowledge, enabling deeper inference across multiple sources [4]. The improvement in Joint metrics highlights the system’s strength in complex query scenarios, where simple document retrieval alone is insufficient. This confirms that the knowledge graph expansion and evidence fusion stages play a critical role in achieving robust multi-hop reasoning [8, 16].

Table 7.3: Joint Metrics

Metric	Baseline	Proposed KG-RAG
Joint EM	0.0000	0.0027
Joint F1	0.2173	0.3357 (+54.50%)
Joint Precision	0.1439	0.2823 (+96.18%)

Chapter 8

CONCLUSIONS AND FUTURE SCOPE

This chapter summarizes the key outcomes of the proposed KG-RAG system and reflects on the effectiveness of the implemented approach. It also outlines potential directions for future enhancement to improve scalability, reasoning capability, and real-world applicability [16, 11].

8.1 Conclusion

This project implemented a Knowledge Graph–Enhanced Retrieval-Augmented Generation (KG-RAG) framework for multi-hop question answering by tightly integrating dense document retrieval [12], knowledge graph expansion [7], graph-aware reranking [14], and evidence-grounded answer generation [2]. The system was evaluated on the HotpotQA distractor dataset [3] using standard metrics such as Exact Match (EM), F1, supporting fact precision, and joint reasoning accuracy.

Experimental results demonstrate that the proposed KG-RAG model significantly outperforms the baseline RAG approach. Knowledge graph expansion improved supporting fact precision by **+33.0%**, while reranking and deduplication further contributed **+30.9%** and **+11.2%** gains, resulting in an overall improvement of **+86.48%** in supporting fact precision. The model also achieved higher Answer EM (**+2.31%**) and F1 (**+0.93%**), indicating better exact matching and token-level answer quality [18].

The results confirm that incorporating structured knowledge graphs enables more reliable multi-hop reasoning, reduces hallucinated outputs [6], and provides verifiable supporting evidence alongside generated answers [9]. Overall, the proposed KG-RAG framework demonstrates a robust and explainable approach to knowledge-intensive question answering and is well-suited for applications in research assistance, educational systems, and enterprise knowledge management [1, 17].

8.2 Future Scope

Although the proposed KG-RAG framework demonstrates significant improvements in multi-hop reasoning and answer grounding, several extensions can further enhance its performance and applicability.

First, the system can be extended to support **dynamic and real-time knowledge graph updates** [4]. Currently, the knowledge graph is constructed from a fixed subset of Wikidata. Incorporating continuously updated knowledge sources and automatic entity ingestion would enable the model to answer time-sensitive and evolving queries more accurately.

Second, future work can explore **adaptive multi-hop reasoning strategies** [8]. The current system performs fixed-hop KG expansion; however, learning the optimal hop depth based on query complexity could reduce noise and improve both retrieval precision and computational efficiency.

Third, the framework can be enhanced with **domain-specific knowledge graphs** for specialized applications such as healthcare, law, finance, and technical documentation [21, 22]. Integrating curated domain ontologies would further improve answer correctness, reduce hallucinations, and increase trustworthiness in high-stakes decision-support systems.

Additionally, future research can incorporate **explainability and user-interaction mechanisms**, such as visualizing reasoning paths over the knowledge graph and allowing users to inspect or modify retrieved evidence [9]. This would improve transparency and user confidence in system outputs.

Finally, large-scale benchmarking on additional multi-hop datasets and deployment in real-world environments can help evaluate scalability, latency, and robustness, paving the way for production-level knowledge-grounded AI systems [11].

REFERENCES

- [1] Y. et al. Wu. A survey on question answering over knowledge graphs. *IEEE Transactions on Knowledge and Data Engineering*, 34(4):1821–1846, 2020.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, M. Lewis, W. Yih, T. Rocktäschel, and S. Riedel. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *NeurIPS*, pages 5479–5494, 2020.
- [3] Z. et al. Yang. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, pages 2369–2380, 2018.
- [4] P. Bin and P. Yun. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024.
- [5] K. VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Psychologist*, 46(4):197–221, 2011.
- [6] G. Izacard and E. Grave. Leveraging passage retrieval with generative models for open-domain question answering. In *Proceedings of EACL*, pages 874–880, 2021.
- [7] Y. et al. Hao. Graph-based multi-hop question answering: A survey. *Artificial Intelligence Review*, 55(1):857–900, 2021.
- [8] D. et al. Li. Knowledge graph-enhanced large language model for domain-specific question answering. *IEEE Access*, 2024.
- [9] W. et al. Xie. Webow-rag: An adaptive approach for retrieval-augmented generation integrating web search and knowledge graphs. In *Proceedings of KDD*, 2024.
- [10] L. W. Anderson and D. R. Krathwohl. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives*. Longman, 2001.
- [11] X. Zhu, Y. Xie, Y. Liu, and Y. Hu. Knowledge graph-guided retrieval augmented generation. *arXiv preprint arXiv:2502.06864*, 2025.
- [12] V. Karpukhin, B. Oguz, S. Min, P. Lewis, P. Wu, S. Edunov, D. Chen, and W. Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, pages 6769–6781, 2020.
- [13] L. et al. Jing. Knowledge graph-based educational recommendation systems: A survey. In *Proceedings of EDM*, pages 220–231, 2021.

-
- [14] R. Nogueira and K. Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.
 - [15] O. Khattab and M. Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. *Proceedings of SIGIR*, pages 39–48, 2020.
 - [16] C. H. Chen and S. W. M. F. Kage. A knowledge-graph-enhanced rag for difficulty-controlled question generation. *arXiv preprint arXiv:2505.07618*, 2025.
 - [17] C. Dong, S. Chen, S. Yuan, Y. Wen, and W. Zhou. How to build an adaptive ai tutor using knowledge graph enhanced rag. In *Proceedings of ICEIT*, 2025.
 - [18] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.
 - [19] B. et al. Goodrich. Transforming complex questions into templates for question answering. *Proceedings of ACL*, pages 1876–1888, 2019.
 - [20] J. Devlin, M. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4182, 2019.
 - [21] H. Kurinjala, M. Kleeding, and A. Ekelhart. Cykg-rag: Knowledge-graph enhanced retrieval augmented generation for cybersecurity. In *Proceedings of ISWC*, 2024.
 - [22] N. et al. Matsumoto. Kragen: A knowledge graph-enhanced rag framework for biomedical problem solving. *Bioinformatics*, 40(6), 2024.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

▸ Quoted Text

Match Groups

-  **23 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 3%  Publications
- 4%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.